

Al-Nahrain University
College of Engineering
Department of Computer
Fourth Stage/First Semester



Microcontrollers Lab

Experiment No.3: Traffic Light Control Using 80C51 Microcontroller

Course Lecture : Mustafa Adnan

Laboratory Assistant: Sarah Haider

Name of student: Aisha Ammar Abdalrhman

Date: 2025/10/30

Objective:

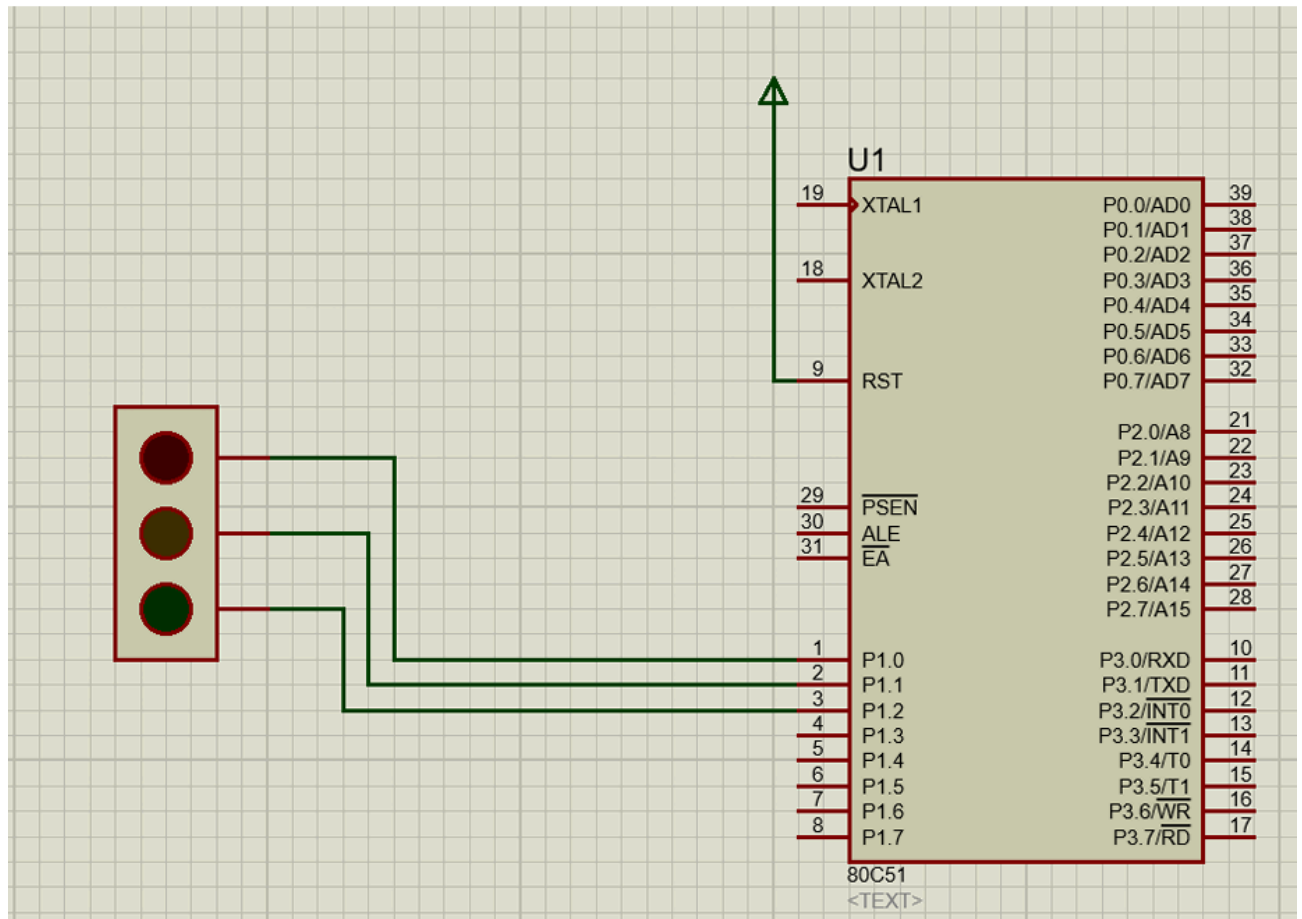
To design and implement a traffic light control system using the 80C51 microcontroller. The experiment will involve writing a C program that simulates a traffic light system with red, yellow, and green lights.

Materials Needed:

- 80C51 Microcontroller
- Keil C Compiler
- LCD/LED setup for visual representation (optional)
- Breadboard and jumper wires
- Power supply

Circuit Diagram :

(Include a circuit diagram showing connections of the microcontroller to LEDs representing red, yellow, and green lights.)



Procedure:

1. The circuit was set up according to the circuit diagram.
2. The LEDs (or other indicators) were connected to the specified pins of the 80C51 microcontroller.
3. The provided C program was loaded into the Keil IDE.
4. The program was compiled and was uploaded to the microcontroller.
5. The system was powered on, and the operation of the traffic lights was observed.

Discussion Questions :

1. What is the purpose of the `sbit` keyword in the code?

The `sbit` keyword is used to assign a name to a specific bit in a special function register (SFR) of the microcontroller. It allows direct control of individual pins or bits, such as turning specific LEDs ON or OFF, which simplifies code readability and manipulation.

2. What role does the delay function serve in the program?

The delay function creates a time interval between switching the traffic lights. It ensures that each light (red, yellow, green) stays ON for a specific duration, allowing the sequence to mimic real-world traffic light timing.

3. How does the microcontroller control the traffic light states?

The microcontroller controls the traffic lights by setting and clearing specific output pins connected to LEDs. Through programmed logic and timing delays, each LED (representing a light) is activated in sequence to simulate a traffic control cycle.

4. What would happen if the delay values were changed?

Changing the delay values would alter the duration of each light's ON period. Increasing the delay would make each light stay ON longer, while decreasing it would make the lights change faster, affecting the overall traffic timing.

5. Why is it necessary to turn off the lights before turning on the next one?

Turning off the lights before activating the next one prevents overlapping signals, which could confuse drivers or users. It ensures that only one signal (red, yellow, or green) is visible at a time, maintaining clear and safe traffic control.

6. What would be the effect of removing the yellow light phase?

Removing the yellow light phase would eliminate the transition warning between the red and green lights. This could lead to unsafe or abrupt traffic changes, as drivers would not receive a visual cue to prepare for stopping or moving.

7 – Can this code be modified for a more complex system?

Yes, the code can be modified to control multiple traffic lights, handle pedestrian crossings, or even adjust timing automatically based on sensors.

You can achieve this by:

- Using more ports (e.g., P1 for North–South, P2 for East–West).
- Using timers instead of delay loops for precise timing.
- Introducing conditions and state-based logic for complex sequences.

```

#include <reg51.h>

// Define LEDs for North-South direction
sbit RED_NS = P1^0;
sbit YELLOW_NS = P1^1;
sbit GREEN_NS = P1^2;

// Define LEDs for East-West direction
sbit RED_EW = P2^0;
sbit YELLOW_EW = P2^1;
sbit GREEN_EW = P2^2;

void delay(unsigned int time) {
    unsigned int i, j;
    for(i=0; i<time; i++)
        for(j=0; j<1275; j++);
}

void main() {
    while(1) {
        // Step 1: North-South Green, East-West Red
        GREEN_NS = 1; YELLOW_NS = 0; RED_NS = 0;
        RED_EW = 1; YELLOW_EW = 0; GREEN_EW = 0;
        delay(500);

        // Step 2: North-South Yellow
        GREEN_NS = 0; YELLOW_NS = 1; RED_NS = 0;
        RED_EW = 1; YELLOW_EW = 0; GREEN_EW = 0;
        delay(200);

        // Step 3: North-South Red, East-West Green
        GREEN_NS = 0; YELLOW_NS = 0; RED_NS = 1;
        RED_EW = 0; YELLOW_EW = 0; GREEN_EW = 1;
        delay(500);

        // Step 4: East-West Yellow
        GREEN_NS = 0; YELLOW_NS = 0; RED_NS = 1;
        RED_EW = 0; YELLOW_EW = 1; GREEN_EW = 0;
        delay(200);
    }
}

```

8 – How does the main function reflect the operation?

The `while(1)` loop keeps repeating the traffic sequence:

- Red → Yellow → Green → repeat

This structure imitates the continuous and cyclic operation of real traffic lights.

9 – What improvements can be made?

- Use hardware timers instead of software delays.
- Add sensor-based timing (adaptive control).
- Include pedestrian crossing lights or countdown displays.
- Add emergency override or synchronization between intersections.
- Use LCD display for timing or debugging.

10 – Real-world considerations

- Safety: Ensure failsafe default (e.g., flashing red during malfunction).
- Power: Use battery/solar backup.
- Reliability: Use weatherproof components.
- Timing accuracy: Hardware timer or RTC-based control.
- Compliance: Follow traffic regulations for timing and colors.
- Maintenance: Easy replacement and clear indication visibility.